

COOPERATIVA DE AHORRO

# Sistema de Control de Gastos Personales

## Plan de Releases — Gestión de Proyecto

|                       |                                                 |
|-----------------------|-------------------------------------------------|
| Proyecto              | Sistema de Control de Gastos Personales         |
| Tipo                  | Aplicación Web — Cooperativa de Ahorro          |
| Stack                 | NestJS + Next.js + PostgreSQL + Docker + Prisma |
| Versión del Documento | 1.0 — Junio 2025                                |

Programación Avanzada | Documento de Planificación de Software

# 1. Información General

El presente documento describe el plan de releases del Sistema de Control de Gastos Personales, una herramienta web desarrollada para los asociados de una cooperativa de ahorro. Su propósito es brindar a los usuarios una plataforma digital donde puedan registrar y controlar sus ingresos y egresos, definir presupuestos mensuales, visualizar resúmenes financieros y recibir alertas automáticas al superar los límites establecidos.

| Campo             | Detalle                                         |
|-------------------|-------------------------------------------------|
| Proyecto          | Sistema de Control de Gastos Personales         |
| Tipo              | Aplicación Web — Cooperativa de Ahorro          |
| Stack             | NestJS + Next.js + PostgreSQL + Docker + Prisma |
| Repositorio       | cooperativa/control-gastos-sistema              |
| Total de Releases | 2 releases — 5 sprints                          |
| Duración Total    | 10 semanas aproximadas                          |

# 2. Casos de Uso del Proyecto

| #     | Caso de Uso                                                                                                        |
|-------|--------------------------------------------------------------------------------------------------------------------|
| CU-01 | Registrar usuario con datos personales, correo, contraseña y moneda preferida para el seguimiento financiero.      |
| CU-02 | Gestionar categorías de gasto e ingreso (crear, editar, eliminar) para clasificar las transacciones.               |
| CU-03 | Registrar transacciones de tipo ingreso o egreso asociadas a una categoría, período y fecha.                       |
| CU-04 | Definir un presupuesto mensual por categoría y compararlo automáticamente con los gastos registrados.              |
| CU-05 | Consultar el resumen mensual de ingresos, egresos, balance neto y estado de cada presupuesto con alertas visuales. |

# 3. Plan de Releases

El plan se organiza en dos releases principales, cada uno con objetivos claramente delimitados. El Release 1 cubre la construcción del backend completo con arquitectura en capas y el frontend base con operaciones CRUD para todas las entidades. El Release 2 se centra en la integración frontend ↔ backend, los flujos avanzados de presupuesto y alertas, y el despliegue final con Docker.

## Release 1 — Primer Corte: Backend y Frontend Base

*Objetivo: Entregar la API REST completa con arquitectura en capas (Controller → Service → Repository) y el frontend con las vistas CRUD para todas las entidades base del sistema financiero.*

| Sprint                                             | Período       | Festivos                  | Alcance                                                                                                             | Historias de Usuario       |
|----------------------------------------------------|---------------|---------------------------|---------------------------------------------------------------------------------------------------------------------|----------------------------|
| <b>Sprint 1 — Infraestructura y entidades base</b> | Sem 1 → Sem 2 | —                         | Docker Compose, Prisma schema, migraciones, módulos CRUD de Usuario, Categoría y TipoTransaccion                    | <b>HU-01, HU-02, HU-03</b> |
| <b>Sprint 2 — Transacciones y cross-cutting</b>    | Sem 3 → Sem 4 | Variable según calendario | Módulo de Transaccion y Periodo. Common module (Filters, Pipes, Interceptors)                                       | <b>HU-04, HU-05, HU-06</b> |
| <b>Sprint 3 — Presupuestos y Frontend base</b>     | Sem 5 → Sem 6 | —                         | Módulo de Presupuesto con lógica de alertas. Frontend: estructura Next.js, listados y formularios de entidades base | <b>HU-07, HU-08, HU-09</b> |

■ **Cierre Primer Corte:** Al finalizar el Sprint 3 (semana 6)

## Release 2 — Segundo Corte: Integración y Despliegue

*Objetivo: Integración completa frontend ↔ backend, formularios avanzados con relaciones, dashboard de resumen financiero, registro de presupuestos con alertas visuales desde la interfaz y despliegue funcional con Docker.*

| Sprint                                            | Período        | Festivos                  | Alcance                                                                                                                                     | Historias de Usuario |
|---------------------------------------------------|----------------|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| <b>Sprint 4 — Frontend avanzado e integración</b> | Sem 7 → Sem 8  | Variable según calendario | Formularios con relaciones (selects dinámicos), dashboard de resumen financiero, estados de carga y error, alertas visuales por presupuesto | <b>HU-10</b>         |
| <b>Sprint 5 — Cierre y despliegue</b>             | Sem 9 → Sem 10 | Variable según calendario | Integración de flujos completos (registrar → presupuestar → resumir), pruebas de integración, despliegue con Docker Compose, README         | <b>HU-11</b>         |

- **Cierre Segundo Corte:** Al finalizar el Sprint 5 (semana 10)

## 4. Historias de Usuario

---

### HU-01 — Gestión de Usuarios (CU-01)

Como administrador de la cooperativa, quiero registrar, consultar, editar y eliminar usuarios asociados, para mantener actualizada la información de los miembros que utilizarán el sistema.

#### Criterios de Aceptación

- ☐ Se puede registrar un usuario con: nombres, apellidos, correo electrónico, contraseña cifrada, fecha de nacimiento y moneda preferida.
- ☐ El correo electrónico es único; si se duplica, el sistema retorna un error claro.
- ☐ La contraseña debe cumplir criterios mínimos de seguridad (mínimo 8 caracteres).
- ☐ Se puede consultar la lista de todos los usuarios con paginación.
- ☐ Se puede consultar un usuario por su ID con sus datos completos.
- ☐ Se puede editar la información de un usuario existente.
- ☐ Se puede eliminar (soft delete) un usuario que no tenga transacciones asociadas.
- ☐ Si se intenta eliminar un usuario con transacciones, el sistema retorna un error descriptivo.

#### Tareas Técnicas

- Backend: CreateUsuarioDto, UpdateUsuarioDto con class-validator
- Backend: UsuarioRepository (Prisma queries)
- Backend: UsuarioService (lógica de negocio + cifrado de contraseña con bcrypt)
- Backend: UsuarioController (endpoints GET, GET/:id, POST, PUT/:id, DELETE/:id)
- Frontend: Página de listado /usuarios
- Frontend: Formulario de creación /usuarios/new
- Frontend: Página de detalle /usuarios/[id]

### HU-02 — Gestión de Categorías (CU-02)

Como usuario registrado, quiero crear y gestionar categorías personalizadas de ingreso y gasto, para clasificar correctamente mis transacciones financieras.

#### Criterios de Aceptación

- ☐ Se puede crear una categoría con: nombre, descripción, tipo (INGRESO / EGRESO) e icono opcional.
- ☐ El nombre de la categoría es único por usuario.
- ☐ Se puede listar todas las categorías del usuario autenticado.
- ☐ Se puede editar el nombre, descripción e icono de una categoría.
- ☐ Se puede eliminar una categoría que no tenga transacciones asociadas.
- ☐ Si se intenta eliminar una categoría con transacciones, el sistema retorna un error descriptivo.

#### Tareas Técnicas

- Backend: DTOs, Repository, Service, Controller del módulo Categoría

- Backend: Validación de unicidad por usuario en el Service
- Frontend: Listado de categorías con indicador de tipo (ingreso/egreso)
- Frontend: Formulario de creación/edición de categoría

### HU-03 — Gestión de Tipos de Transacción y Períodos (Soporte CU-03) (CU-02, CU-03)

Como administrador, quiero configurar los tipos de transacción y los períodos mensuales del sistema, para organizar temporalmente las transacciones, presupuestos y resúmenes financieros.

#### Criterios de Aceptación

- ☐ Existen al menos dos tipos de transacción predefinidos: INGRESO y EGRESO.
- ☐ Se puede crear un período con: nombre (ej: '2025-06'), fecha de inicio y fecha de fin.
- ☐ El nombre del período es único.
- ☐ La fecha de fin debe ser posterior a la fecha de inicio.
- ☐ Solo puede existir un período activo a la vez.
- ☐ Se puede listar todos los períodos ordenados por fecha.
- ☐ Se puede editar y cambiar el estado de un período.

#### Tareas Técnicas

- Backend: Seed de TipoTransaccion en Prisma
- Backend: DTOs, Repository, Service, Controller del módulo Periodo
- Backend: Validación de período activo único en el Service
- Frontend: Listado de períodos con indicador de estado activo/inactivo

### HU-04 — Registro de Transacciones (CU-03)

Como usuario registrado, quiero registrar mis ingresos y egresos con su respectiva categoría, monto, fecha y descripción, para llevar un historial detallado de mis movimientos financieros.

#### Criterios de Aceptación

- ☐ Se puede registrar una transacción con: monto, fecha, descripción, categoríaId, períodoId y tipoTransaccionId.
- ☐ El monto debe ser un valor positivo mayor a cero.
- ☐ La categoría, el período y el tipo de transacción deben existir previamente.
- ☐ La categoría seleccionada debe corresponder al tipo de transacción (INGRESO/EGRESO).
- ☐ Se puede listar las transacciones del usuario con filtros por período y categoría.
- ☐ Se puede editar y eliminar una transacción propia.

#### Tareas Técnicas

- Backend: DTO con validación de FKs (categoríaId, períodoId, tipoTransaccionId)
- Backend: Validación de coherencia tipo-categoría en el Service
- Backend: TransaccionRepository, TransaccionService, TransaccionController
- Frontend: Formulario con selects dinámicos (tipo → categoría → período)
- Frontend: Listado de transacciones con filtros y paginación

## HU-05 — Gestión de Presupuestos Mensuales (CU-04)

Como usuario registrado, quiero definir un presupuesto mensual por categoría, para planificar mis finanzas y controlar que mis gastos no superen los límites establecidos.

### Criterios de Aceptación

- ☐ Se puede crear un presupuesto seleccionando: usuario, categoría, período y monto límite.
- ☐ No se permite duplicar el presupuesto para la misma categoría en el mismo período.
- ☐ El monto límite debe ser un valor positivo.
- ☐ La categoría y el período deben existir previamente.
- ☐ Se puede listar los presupuestos del usuario filtrados por período.
- ☐ Se puede editar y eliminar un presupuesto existente.

### Tareas Técnicas

- Backend: DTO con validación de unicidad compuesta (usuariold + categoríald + periodold)
- Backend: PresupuestoRepository, PresupuestoService, PresupuestoController
- Frontend: Formulario con selects de categoría y período
- Frontend: Listado de presupuestos con indicador de uso vs límite

## HU-06 — Comparación Presupuesto vs. Gastos y Alertas (CU-04)

Como usuario registrado, quiero visualizar el porcentaje de uso de cada presupuesto y recibir una alerta cuando mis gastos superen el límite definido, para tomar decisiones financieras oportunas.

### Criterios de Aceptación

- ☐ El sistema calcula automáticamente el total gastado por categoría en un período.
- ☐ El porcentaje de uso se calcula como  $(total\_gastado / monto\_límite) \times 100$ .
- ☐ Si el porcentaje supera el 100%, el sistema genera una alerta de presupuesto excedido.
- ☐ Si el porcentaje supera el 80%, el sistema genera una alerta de advertencia.
- ☐ Las alertas se reflejan visualmente en la interfaz (colores: verde < 80%, amarillo 80-100%, rojo > 100%).
- ☐ Se puede consultar el estado de todos los presupuestos del período activo en un solo endpoint.

### Tareas Técnicas

- Backend: Lógica de cálculo de uso en PresupuestoService
- Backend: Endpoint GET /presupuestos/estado/:periodold con datos calculados
- Frontend: Componente de barra de progreso por presupuesto
- Frontend: Sistema de alertas visuales con código de color

## HU-07 — Resumen Financiero Mensual (CU-05)

Como usuario registrado, quiero consultar un resumen mensual de mis ingresos, egresos, balance neto y estado de presupuestos, para evaluar mi salud financiera de forma rápida y clara.

### Criterios de Aceptación

- ☐ Se puede consultar el resumen de un período seleccionado.
- ☐ El resumen incluye: total de ingresos, total de egresos y balance neto del período.
- ☐ El resumen incluye el desglose de gastos por categoría con su participación porcentual.
- ☐ Se muestra el estado de cada presupuesto (cumplido, en alerta, excedido).
- ☐ El resumen se puede consultar para cualquier período histórico del usuario.

### Tareas Técnicas

- Backend: Endpoint GET /resumen/:periodoId con cálculos agregados
- Backend: Queries de agregación en Prisma (sum, groupBy)
- Frontend: Página /resumen con tarjetas de totales
- Frontend: Gráfica de barras o donut por categoría (usando recharts o similar)
- Frontend: Tabla de estado de presupuestos en el resumen

## HU-08 — Common Module: Filtros, Interceptores y Pipes (Cross-cutting)

Como desarrollador, quiero implementar un módulo compartido con filtros de excepción, interceptores de respuesta y pipes de validación, para garantizar respuestas consistentes y manejo centralizado de errores en toda la API.

### Criterios de Aceptación

- ☐ Todas las excepciones HTTP retornan un JSON con formato uniforme: { statusCode, message, error, timestamp }.
- ☐ Todas las respuestas exitosas retornan formato uniforme: { statusCode, message, data }.
- ☐ El ValidationPipe global está configurado con whitelist: true y forbidNonWhitelisted: true.
- ☐ Los errores de validación retornan mensajes claros indicando qué campo falló y por qué.

### Tareas Técnicas

- common/filters/http-exception.filter.ts
- common/interceptors/response.interceptor.ts
- common/pipes/validation.pipe.ts
- Registro global en main.ts

## HU-09 — Autenticación de Usuarios (JWT) (CU-01)

Como usuario registrado, quiero iniciar y cerrar sesión de forma segura usando JWT, para que solo yo pueda acceder a mi información financiera.

### Criterios de Aceptación

- ☐ El usuario puede iniciar sesión con correo y contraseña y recibir un JWT válido.
- ☐ El JWT tiene una expiración configurada (ej. 24 horas).
- ☐ Los endpoints protegidos retornan 401 si no se envía un token válido.
- ☐ El usuario puede cerrar sesión (invalidar el token desde el cliente).
- ☐ El frontend almacena el token de forma segura y lo adjunta a cada petición.



### Tareas Técnicas

- Backend: AuthModule con JwtStrategy y PassportJS
- Backend: AuthController con endpoints POST /auth/login y POST /auth/register
- Backend: Decorador @CurrentUser() para extraer el usuario del token
- Frontend: Contexto de autenticación global (AuthContext)
- Frontend: Middleware de protección de rutas en Next.js
- Frontend: Página /login con formulario y manejo de errores

## HU-10 — Frontend: Dashboard, Formularios y Navegación (CU-01 a CU-05)

Como usuario del sistema, quiero gestionar mis transacciones, categorías, presupuestos y consultar mi resumen financiero desde la interfaz web, para administrar mis finanzas sin depender de herramientas externas.

### Criterios de Aceptación

- ☐ Existe un layout general con sidebar o navbar con enlaces a cada sección (Dashboard, Transacciones, Categorías, Presupuestos, Resumen).
- ☐ La navegación indica visualmente la sección activa.
- ☐ El diseño es responsivo (funcional en desktop y tablet).
- ☐ Existen páginas de listado para cada entidad con datos provenientes de la API.
- ☐ Existen formularios de creación/edición con validación del lado del cliente.
- ☐ Los formularios con relaciones usan selects dinámicos (ej: tipo → categoría).
- ☐ Los formularios muestran mensajes de error claros si alguna validación falla.
- ☐ Los formularios muestran estado de carga (spinner/skeleton) durante las peticiones.
- ☐ Tras una operación exitosa, se muestra un mensaje de confirmación y se actualizan los listados.
- ☐ El dashboard muestra un resumen del período activo con alertas de presupuesto.

### Tareas Técnicas

- Frontend: layout.tsx con componente de navegación (Sidebar o Navbar)
- Frontend: Cliente HTTP centralizado (lib/api.ts) y services/ por entidad
- Frontend: Páginas de listado para cada entidad
- Frontend: Formularios de creación/edición con validación
- Frontend: Página /dashboard con resumen del período activo
- Frontend: Componentes de feedback (toast/alert de éxito/error)
- Frontend: Estilos responsivos con CSS/Tailwind

## HU-11 — Integración Final y Despliegue con Docker (Transversal)

Como equipo de desarrollo, quiero verificar que todos los servicios funcionen correctamente de forma integrada, para garantizar que el sistema se despliega con un solo comando y los flujos completos funcionan de extremo a extremo.

### Criterios de Aceptación

- ☐ Los 3 servicios (PostgreSQL, NestJS, Next.js) se levantan correctamente con docker compose up.

- ☐ El frontend se comunica correctamente con el backend y muestra datos reales de la base de datos.
- ☐ El flujo completo funciona: registrar usuario → crear categoría → registrar transacción → definir presupuesto → consultar resumen.
- ☐ No hay errores de consola ni advertencias críticas en backend ni frontend.
- ☐ Las migraciones de Prisma se aplican correctamente al iniciar el contenedor.
- ☐ El README.md del repositorio incluye instrucciones claras de instalación y ejecución.

### **Tareas Técnicas**

- Verificar docker-compose.yml con healthchecks y dependencias correctas
- Pruebas de integración del flujo completo
- Documentación del README.md (descripción, tecnologías, instalación, ejecución)
- Revisión de código y limpieza final

## 5. Definition of Done (DoD) Global

Cada Historia de Usuario se considera terminada cuando cumple todos los siguientes criterios:

### Backend

- ☐ Endpoint(s) implementados con arquitectura en capas: Controller → Service → Repository.
- ☐ DTOs con validaciones usando class-validator y class-transformer.
- ☐ Manejo de errores con excepciones HTTP apropiadas (NotFoundException, ConflictException, BadRequestException).
- ☐ Respuestas con formato uniforme (interceptor aplicado).
- ☐ Endpoint probado manualmente con Postman/Thunder Client y funcionando correctamente.

### Frontend

- ☐ Página(s) implementada(s) con componentes reutilizables.
- ☐ Consumo del API a través de la capa de services/.
- ☐ Manejo de estados: carga (loading), éxito y error.
- ☐ Formularios con validación del lado del cliente.
- ☐ Diseño responsivo y navegable.

### Infraestructura y Código

- ☐ Código versionado en GitHub con commits descriptivos.
- ☐ El servicio funciona correctamente con docker compose up.
- ☐ No hay errores de consola ni advertencias críticas.
- ☐ Las migraciones de Prisma están aplicadas y el esquema es consistente.

## 6. Entidades del Modelo de Datos (Prisma)

### Diagrama de Relaciones

| Entidad Origen | Cardinalidad | Entidad Destino | Descripción                                  |
|----------------|--------------|-----------------|----------------------------------------------|
| Usuario        | 1 → N        | Transaccion     | Un usuario registra múltiples transacciones. |
| Usuario        | 1 → N        | Presupuesto     | Un usuario define múltiples presupuestos.    |

| Entidad Origen  | Cardinalidad | Entidad Destino | Descripción                                        |
|-----------------|--------------|-----------------|----------------------------------------------------|
| Usuario         | 1 → N        | Categoria       | Un usuario crea y gestiona sus propias categorías. |
| Categoria       | 1 → N        | Transaccion     | Una categoría agrupa múltiples transacciones.      |
| Categoria       | 1 → N        | Presupuesto     | Una categoría puede tener presupuesto por período. |
| TipoTransaccion | 1 → N        | Transaccion     | Clasifica si la transacción es ingreso o egreso.   |
| Periodo         | 1 → N        | Transaccion     | Un período agrupa transacciones del mes.           |
| Periodo         | 1 → N        | Presupuesto     | Un período define el alcance de los presupuestos.  |

## Resumen de Entidades

| Entidad                | Campos Principales                                                                                        |
|------------------------|-----------------------------------------------------------------------------------------------------------|
| <b>Usuario</b>         | id, nombres, apellidos, correoElectronico (unique), passwordHash, fechaNacimiento, monedaPreferida        |
| <b>Categoria</b>       | id, nombre (unique por usuario), descripcion, tipo (INGRESO/EGRESO), icono, usuariold                     |
| <b>TipoTransaccion</b> | id, nombre (unique): INGRESO   EGRESO, descripcion                                                        |
| <b>Periodo</b>         | id, nombre (unique), fechaInicio, fechaFin, activo (boolean)                                              |
| <b>Transaccion</b>     | id, monto, fecha, descripcion, usuariold, categoriold, periodold, tipoTransaccionId                       |
| <b>Presupuesto</b>     | id, montoLimite, usuariold, categoriold, periodold (unique compound: usuariold + categoriold + periodold) |

## 7. Cronograma de Releases

### Release 1 — Primer Corte — Backend + Frontend Base

| Sprint          | Período     | Alcance                                             | Festivos |
|-----------------|-------------|-----------------------------------------------------|----------|
| <b>Sprint 1</b> | Semanas 1–2 | Docker, Prisma, Usuario, Categoría, TipoTransaccion | —        |

| Sprint          | Período     | Alcance                                                               | Festivos         |
|-----------------|-------------|-----------------------------------------------------------------------|------------------|
| <b>Sprint 2</b> | Semanas 3–4 | Transaccion, Periodo, Filters/Pipes, Common Module                    | Según calendario |
| <b>Sprint 3</b> | Semanas 5–6 | Presupuesto, alertas, Auth JWT, Frontend base: listados y formularios | —                |

## Release 2 — Segundo Corte — Integración + Despliegue

| Sprint          | Período      | Alcance                                                                                     | Festivos         |
|-----------------|--------------|---------------------------------------------------------------------------------------------|------------------|
| <b>Sprint 4</b> | Semanas 7–8  | Frontend avanzado: matrículas, presupuestos, dashboard, alertas visuales, selects dinámicos | Según calendario |
| <b>Sprint 5</b> | Semanas 9–10 | Integración de flujos completos, pruebas de integración, Docker Compose, README             | Según calendario |

## 8. Estado del Proyecto

- ☒ Plan de releases revisado y aprobado
- ☒ Historias de Usuario revisadas y aprobadas
- ☒ Criterios de Aceptación revisados y aprobados
- ☒ Definition of Done revisado y aprobado
- ☒ Modelo de datos revisado y aprobado
- ☐ Repositorio GitHub creado con Issues y Milestones

■ **Repositorio:** [github.com/cooperativa/control-gastos-sistema](https://github.com/cooperativa/control-gastos-sistema) ■ **Issues:** 11 HUs + 1 DoD (pinned) ■ **Milestones:** 5 sprints con fechas de cierre